Apache Parquet Explained: A Guide for Data Professionals

This in-depth guide to Apache Parquet breaks it down with clear explanations, and hands-on code examples!

February 2025 13 min read

Laiba Siddiqui
Tech Writer and Editor

Thalia Barrera
Senior Data Science Editor at DataCamp

Big data can be overwhelming — we often feel there's no easy way to manage it. But in reality, Apache Parquet makes this so much easier. It's a smart data storage format that handles large datasets by saving you time and resources.

In this article, I'll walk you through what makes Parquet a unique tool and how to use it in your projects. By the end, you'll know why it is a top choice for data professionals and how to start using it with tools like Python and Spark.

What is Apache Parquet?

Apache Parquet is an open-source columnar storage format that addresses big data processing challenges. Unlike traditional row-based storage, it organizes data into columns. This structure allows you to read only the necessary columns, making data queries faster and reducing resource consumption.

For example, you can target the relevant data instead of processing an entire dataset to find a single attribute. That's a primary reason (among many others, as we will see later) why Parquet is a natural fit for big data frameworks like Apache **Hadoop**, **Apache Spark**, and Apache Hive.

Apart from frameworks, Parquet is also widely used in data lakes and analytics platforms. Teams use Amazon S3, Azure Data Lake Storage, or Google Cloud Storage to store large-scale datasets in a data lake. Since Parquet is optimized for efficient querying, it's a preferred format for storing structured and semi-structured data.

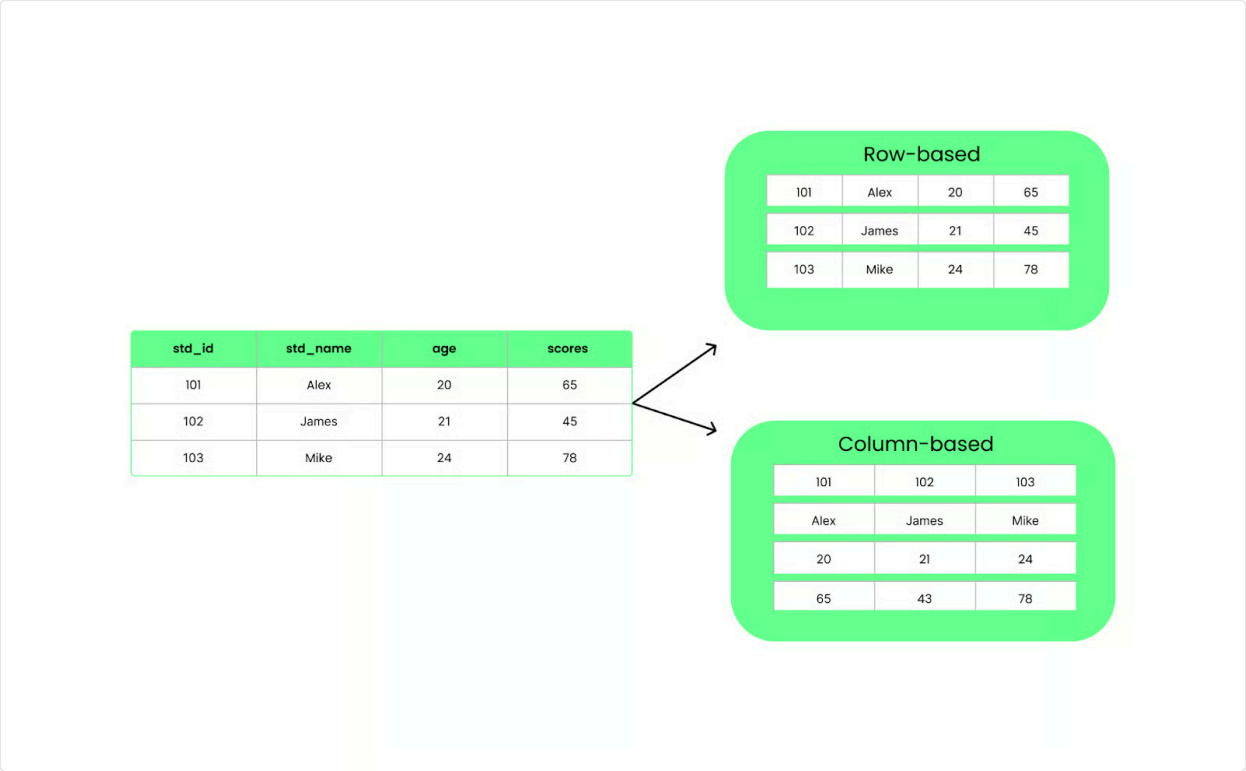
For example, in Amazon S3, a typical workflow might involve using **AWS Glue** to catalog Parquet files and **Amazon Athena** to run SQL queries without loading data into a database.

Features of Apache Parquet

Let’s understand the architecture of Apache Parquet with its key features:

Columnar storage

Unlike row-based formats like CSV, Parquet organizes data in columns. This means when we run a query, it only pulls the specific columns we need instead of loading everything. This improves performance and reduces I/O usage.

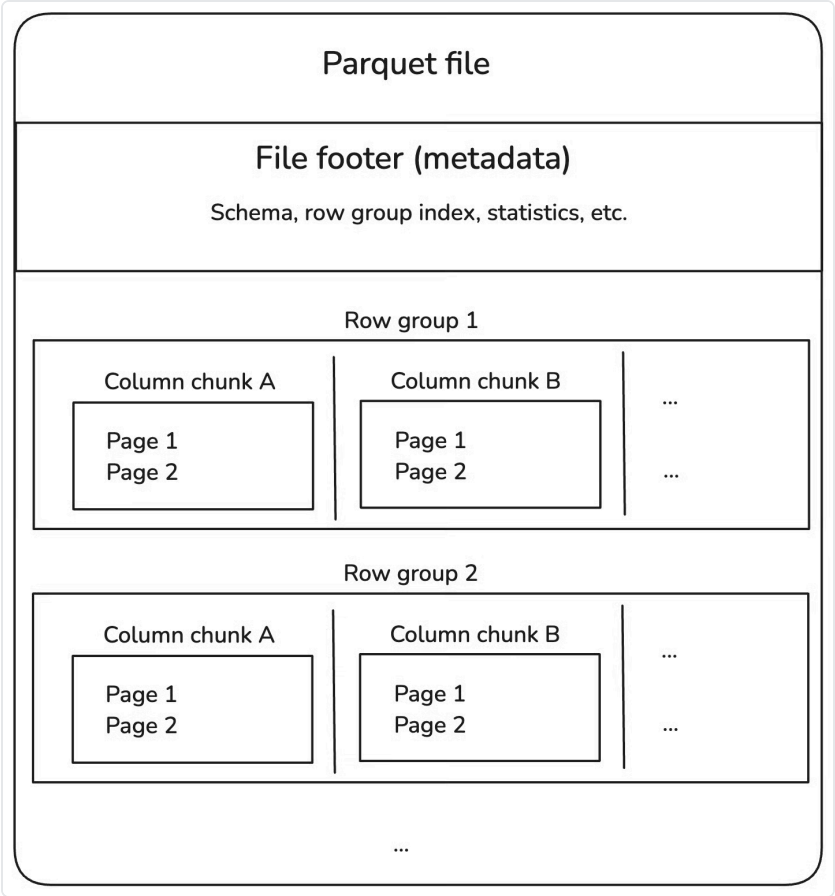


Row vs column-based structure. Image by Author.

Parquet files are split into row groups, which hold a batch of rows. Each row group is broken into column chunks, each containing data for one column. These chunks are further divided into smaller pieces called pages, which are compressed to save space.

In addition, Parquet files store extra information in the footer, called metadata, which locates and reads only the data we need.

Here’s what the structure looks like:



Parquet file internal structure. Image by Author.

Let's break down each component in the diagram above.

Row groups

- A row group contains multiple rows but stores data column-wise for efficient reading.
- Example: A dataset with 1 million rows might be split into 10 groups of 100,000 rows each.

Column chunks

- Within each row group, data is separated by columns.
- This design allows columnar pruning, where we can read only the relevant columns instead of scanning the entire file.

Pages

- Each column chunk is further split into pages to optimize memory usage.
- Pages are typically compressed, reducing storage costs.

Footer (metadata)

- The footer at the end of a Parquet file stores index information:
    - **Schema:** Defines data types and column names.
    - **Row group offsets:** Helps locate specific data quickly.

- **Statistics:** Min/max values to enable predicate pushdown (filtering at the storage level).

Compression and encoding

As mentioned, Parquet compresses data column by column using compression methods like **Snappy** and **Gzip**. It also uses two encoding techniques:

- **Run-length encoding** to store repeated values compactly.
- **Dictionary encoding** to replace duplicates with dictionary references.

This reduces file sizes and speeds up data reading, which is especially helpful when you work with big data.

Schema evolution

Schema evolution means modifying the structure of datasets, such as adding or altering columns. It may sound simple, but depending on how your data is stored, modifying the schema can be slow and resource-intensive.
Let's understand this by comparing CSV and Parquet schema evolution.
Suppose you have a **CSV** file with columns like `student_id`, `student_name`, and `student_age`. If you want to add a new `scores` column, you'd have to do the following:

1. Read the entire file into memory.
2. Update the header to include a new column, `scores`.
3. Add a score for each student. This means appending values for all rows (even if they are missing, you may need placeholders like empty strings or `NULL`).
4. Save everything as a new CSV file.

CSV is a simple text-based format with no built-in schema support. This means any change to the structure requires rewriting the entire file, and older systems reading the modified file might break if they expect a different structure!
With **Parquet**, you can add, remove, or update fields without breaking your existing files. As we saw before, Parquet stores schema information inside the file footer (metadata), allowing for evolving schemas without modifying existing files.
Here's how it works:

- When you add a new column, existing Parquet files remain unchanged.
- New files will include the additional column, while old files still follow the previous schema.
- Removing a column doesn't require reprocessing previous data; queries will ignore the missing column.

- If a column doesn't exist in an older file, Parquet engines (like Apache Spark, Hive, or BigQuery) return `NULL` instead of breaking the query.
- Older Parquet files can be read even after schema modifications.
- Newer Parquet files with additional columns can still be read by systems expecting an older schema.

Adding a column to the Parquet file without breaking it. Image by Author.

Language and platform support

Parquet supports different programming languages, such as Java, Python, C++, and Rust. This means developers can easily use it regardless of their platform. It is also natively integrated with big data frameworks like Apache Spark, Hive, Presto, Flink, and Trino, ensuring efficient data processing at scale.
So whether you're using Python (through PySpark) or another language, Parquet can manage the data in a way that makes it easy to query and analyze across different platforms.
If you're new to big data frameworks, I recommend taking the **Introduction to PySpark** course. It's a great way to get started.

How to Read and Write Parquet Files

Now that you know the basics of Apache Parquet, I'll walk you through writing, reading, and integrating Parquet files with **pandas**, PyArrow, and other big data frameworks like Spark.

Write Parquet files using pandas

To save DataFrames as Parquet files, you need pandas and a Parquet engine like PyArrow:

```
pip install pandas pyarrow
```

Now, write a Parquet file using the following code:

```
import pandas as pd

# Sample DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}
df = pd.DataFrame(data)

# Write to Parquet file
df.to_parquet("data.parquet", engine="pyarrow", index=False)

print("Parquet file written successfully!")
```

```
Parquet file written successfully!
```

Write the Parquet file with pandas. Image by Author.

Read Parquet files using pandas

Here's a simple code to read your Parquet file:

```
import pandas as pd

# Read the Parquet file
df = pd.read_parquet("data.parquet", engine="pyarrow")

print("Data from Parquet file:")
print(df)
```

```
Data from Parquet file:
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
```

Read the Parquet file with pandas. Image by Author.

Write Parquet files using PyArrow

PyArrow is a tool from the Apache Arrow project that makes it easy to work with Parquet files. Here's how you can write a parquet file using PyArrow:

```
import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq

# Sample data
df = pd.DataFrame({
    "Name": ["Jacob", "Lauren", "Oliver"],
    "Age": [25, 30, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
})

# Convert to a PyArrow table
table = pa.Table.from_pandas(df)

# Write to Parquet file
pq.write_table(table, "data.parquet")

print("Parquet file written successfully!")
```

```
Parquet file written successfully!
```

Write a Parquet file with PyArrow. Image by Author.

Read Parquet files using PyArrow

Here's how to read a Parquet file using PyArrow:

```python
import pyarrow.parquet as pq

# Read the Parquet file
table = pq.read_table("data.parquet")

# Convert to a pandas DataFrame
df = table.to_pandas()

print("Data from Parquet file:")
print(df)
```

```
Data from Parquet file:
     Name  Age         City
0   Jacob   25     New York
1  Lauren   30  Los Angeles
2  Oliver   35      Chicago
```

Read the Parquet file with PyArrow. Image by Author.

Integrate with big data frameworks

We can use Spark to read and write Parquet files directly. Download it from **Apache Spark's website** or set it up following the instructions.
Once done, import the libraries and create a DataFrame:

```python
from pyspark.sql import SparkSession

# Initialize a Spark session
spark = SparkSession.builder.appName("SparkExample").getOrCreate()

# Define the schema for the dataset
schema = ["Name", "Age", "City"]

# Create a sample data
data = [
    ("Jacob", 30, "New York"),
    ("Lauren", 35, "Los Angeles"),
    ("Billy", 25, "Chicago")
]

# Create a DataFrame from the sample data
df = spark.createDataFrame(data, schema)

# Show the DataFrame
df.show()
```

```
+------+---+-----------+
|  Name|Age|       City|
+------+---+-----------+
| Jacob| 30|   New York|
|Lauren| 35|Los Angeles|
| Billy| 25|    Chicago|
+------+---+-----------+
```

Create a sample DataFrame in Spark. Image by Author.

Next, write this DataFrame as a Parquet file:

```python
# Write DataFrame to Parquet
df.write.parquet("data.parquet")
```

The `write.parquet()` saves the DataFrame in the Parquet format, and the file will be called `employee.parquet`. Now, to read this Parquet file, you can use the following code:

```
# Read the Parquet file
parquet_df = spark.read.parquet("data.parquet")

# Show the DataFrame
parquet_df.show()
```

```
+------+---+-----------+
|  Name|Age|       City|
+------+---+-----------+
|Lauren| 35|Los Angeles|
| Billy| 25|    Chicago|
| Jacob| 30|   New York|
+------+---+-----------+
```

Read the Parquet file. Image by Author.

Apart from Spark, Parquet can also work with Hive. When you create a Hive table, use `STORED AS PARQUET` to make Parquet the storage format.

Useful Operations with Parquet

Apart from reading and writing, there are some basic operations every developer should know since they are useful when working with Parquet files. Let's review them in this section.
I will use pandas and PyArrow to illustrate the concepts.

Append data to an existing Parquet file

Appending data is useful when new records need to be added without rewriting the entire dataset.

```
import pyarrow.parquet as pq
import pyarrow as pa

# Load existing Parquet file
existing_table = pq.read_table("data.parquet")

# New data
new_data = pd.DataFrame({
    "Name": ["David", "Emma"],
    "Age": [40, 28],
    "City": ["San Francisco", "Seattle"]
})

# Convert new data to PyArrow table
new_table = pa.Table.from_pandas(new_data)

# Concatenate both tables
merged_table = pa.concat_tables([existing_table, new_table])

# Write back to Parquet file
pq.write_table(merged_table, "data.parquet")
```

Read only specific columns from a Parquet file

Instead of loading the entire dataset, you can select only the necessary columns, reducing memory usage and improving performance. This is significantly faster than reading the full dataset:

```
df = pd.read_parquet("data.parquet", columns=["Name", "Age"])
print(df)
```

Filtering data while reading (predicate pushdown)

Parquet allows efficient filtering at the storage level, known as predicate pushdown, which prevents loading unnecessary data. This avoids scanning the entire file, making queries much faster:

```
import pyarrow.parquet as pq

# Read only rows where Age > 30
table = pq.read_table("data.parquet", filters=[("Age", ">", 30)])

df = table.to_pandas()
print(df)
```

Merge multiple Parquet files

Often, Parquet files are stored as separate partitions. You can merge them into a single Parquet file. This is useful when combining datasets from different sources:

```
import pyarrow.parquet as pq
import pyarrow as pa

# List of Parquet files to merge
file_list = ["data_part1.parquet", "data_part2.parquet"]

# Read all files and merge
tables = [pq.read_table(f) for f in file_list]
merged_table = pa.concat_tables(tables)

# Write merged Parquet file
pq.write_table(merged_table, "merged_data.parquet")
```

Convert CSV to Parquet

If you have existing CSV files, converting them to Parquet saves space and speeds up processing, which drastically reduces file size and improves read performance:

```
df = pd.read_csv("data.csv")
df.to_parquet("data.parquet", engine="pyarrow", index=False)
```

Partition Parquet files for faster queries

Partitioning organizes data into subdirectories based on a column value, making queries significantly faster.
Here's how you can write partitioned data:

```
df.to_parquet("partitioned_data/", engine="pyarrow", partition_cols=["Cit
```

The above code creates subdirectories:

```
partitioned_data/City=New York/
partitioned_data/City=Los Angeles/
partitioned_data/City=Chicago/
```

Then, you can read only a specific partition:

```
df = pd.read_parquet("partitioned_data/City=New York/")
print(df)
```

This speeds up analysis by scanning only relevant partitions!

Use compression to optimize storage

Parquet supports compression algorithms like Snappy, Gzip, and Brotli to reduce file size:

```
df.to_parquet("compressed.parquet", engine="pyarrow", compression="snappy
```

Best Practices for Using Apache Parquet

When I first started using Apache Parquet, I realized that minor adjustments could greatly improve its efficiency. Here are some of my top tips for optimizing Parquet in real-world scenarios.

Choose the right compression codec

If you want to save storage, codecs like **Snappy** or **Gzip** can be your go-to options—snappy offers quick compression and decompression, perfect for scenarios where speed matters most.
Conversely, Gzip is ideal if you're tight on storage but can handle slightly slower reads. The key is understanding your workload—a faster codec like Snappy often wins out if you frequently access files. However, Gzip is best for archival data.

Partition data effectively

Break your data into logical subsets, like dividing it by date, region, or any other frequently queried field to reduce the amount of data scanned during a query. I once worked with a dataset containing years of transaction logs and partitioned it by year and month to fetch specific periods in seconds rather than minutes.

Monitor schema evolution

I always ensure that new columns are added in a way that doesn't disrupt existing processes. This usually means appending them rather than modifying existing ones. To do so, you can use Apache Spark's schema evolution support for smoother transitions.

Apache Parquet vs Other Data Formats

Let's compare Parquet with other data storage formats.

Parquet vs CSV

We've already covered this through the blog post, but let me emphasize it again: Parquet and CSV are two different formats that handle data differently.
Parquet organizes data in columns, while CSV arranges it in rows. When you use Parquet, all the data from the same column are grouped together, so you can easily pull data from specific columns without sifting through everything else. It's faster and takes up less space because Parquet compresses data.

Id	name	city
101	Alex	New York
102	Mike	California
103	Bob	Chicago

Parquet column-based format. Image by Author.

CSV, on the other hand, stores data row by row. It's simple and works well for small datasets, but it's not ideal for big ones. Every query has to read the entire row, even if you only need a couple of columns. This slows things down and takes more memory to process.

Row 1	101 Alex New York
Row 2	102 Mike California
Row 3	103 Bob Chicago

CSV row-based format. Image by Author.

Parquet vs JSON

JSON is great for structuring data in a way that's easy to understand, but it has a drawback: it's not very efficient for storage or speed. So, let me explain why Parquet is more efficient than JSON with an example.
Suppose we have an employee data table with three columns: `EmployeeID`, `Department`, and `Location`:

EmployeeID	Department	Location
1	HR	New York
2	HR	New York
3	HR	New York
4	IT	San Francisco
5	IT	San Francisco

Now, if we save this data as JSON, it will look something like this:

```
[
  {"EmployeeID": 1, "Department": "HR", "Location": "New York"},
  {"EmployeeID": 2, "Department": "HR", "Location": "New York"},
  {"EmployeeID": 3, "Department": "HR", "Location": "New York"},
  {"EmployeeID": 4, "Department": "IT", "Location": "San Francisco"},
  {"EmployeeID": 5, "Department": "IT", "Location": "San Francisco"}
]
```

Notice how JSON repeats column names like `EmployeeID`, `Department`, and `Location` for every single record. It also repeats the values for `HR` and `New York` multiple times. This makes the file much larger and slower.
Now let's imagine (because Parquet is not human-readable) we save the same data as Parquet:

- **EmployeeID**: [1, 2, 3, 4, 5]
- **Department**: [HR, HR, HR, IT, IT] (compressed as HR: 3, IT: 2)
- **Location**: [New York, New York, New York, San Francisco, San Francisco] (compressed as New York: 3, San Francisco: 2).

Instead of storing row by row, Parquet organizes the data by columns and compresses repeating values.

Parquet vs Avro

Avro is a row-based format. It's great for tasks like streaming data or processing logs, where you constantly add new records or retrieve complete rows. But Parquet's column-based format

is perfect for analytics. If you run queries to analyze large amounts of data, Parquet would work best. It pulls data from the necessary columns and skips the rest to save time and resources.

In short, Parquet is better for reading and analyzing large datasets, while Avro is ideal for writing and storing data in an easy-to-update way.

Here's a comparison table of Parquet vs. CSV vs. JSON vs. Avro, including pros, cons, and use cases:

Format	Pros	Cons	Use cases
Parquet	✅ Columnar format for fast analytics ✅ High compression efficiency ✅ Supports schema evolution ✅ Optimized for big data frameworks (Spark, Hive, Presto) ✅ Supports predicate pushdown (efficient filtering)	❌ Not human-readable ❌ Slower for row-based operations ❌ More complex write operations	• Big data processing (Hadoop, Spark, Hive) • Cloud data lakes (AWS S3, Azure Data Lake) • Fast analytics and OLAP workloads
CSV	✅ Human-readable and simple ✅ Easy to generate and parse ✅ Compatible with almost all tools	❌ No schema support ❌ Slow for large datasets ❌ Large file sizes (no compression) ❌ Must scan entire file for queries	• Data exchange (simple tabular data) • Small to medium datasets • Interoperability with legacy systems
JSON	✅ Supports nested and semi-structured data ✅ Human-readable ✅ Widely used in web APIs ✅ Flexible schema	❌ Larger file sizes (due to text format) ❌ Slow for big data queries ❌ No native indexing	• Web APIs and NoSQL databases (MongoDB, Elasticsearch) • Streaming applications • Storing logs or semi-structured data
Avro	✅ Row-based format for fast writes ✅ Compact binary format (efficient storage) ✅ Supports schema evolution ✅ Good for streaming and message queues	❌ Not human-readable ❌ Less efficient for analytical queries compared to Parquet ❌ Requires Avro libraries for processing	• Streaming and real-time processing (Kafka, Flink) • Long-term storage with schema changes • Efficient data serialization

When to Use Apache Parquet

In summary, here are a few situations where Parquet is the best choice:

- **Analytics-heavy workloads:** Parquet's columnar format allows us to fetch only the data we need, which speeds up queries and saves time. I've seen this hands-on when processing datasets with Apache Spark — queries that once took minutes ran in seconds because of Parquet's efficient structure.
- **Data lake architectures:** When you build a data lake, storage costs quickly add up. But Parquet's compression capabilities reduce the size of stored data to help you save on storage without sacrificing performance.
- **Use cases involving large datasets:** Parquet handles large and complex datasets neatly, especially those with nested or hierarchical structures. Its support for rich data types and schema evolution ensures you can adapt your data as requirements change.

Final Thoughts

Apache Parquet is perfect for handling big data. It's fast, saves storage space, and works with tools like Spark. If you're excited to learn more, check out **the following resources:**

- **Introduction to Databricks** course for understanding Databricks, a unified data platform to streamline big data workflows.
- **Cleaning Data with PySpark** course for cleaning and preprocessing datasets using PySpark.
- **Big Data with PySpark** track for scaling data processing with Apache Spark using the PySpark API.

TOPICS